HD in Computing & Software Engineering

CSE5015 Computing Project Proposal Approval Sheet

Student Use only;

Group Name	**Team ForeSight**	**Attempt No**	**01**
Title of the project	**SmartRestaurant -** An ML-Based Demand Forecasting and Inventory Decision Support System for Sri Lankan Fast-Food Restaurants		
System platform	☒ Web Application ☐ Mobile Application	☐ Desktop Application ☐ ……	
Technology	The proposed system will incorporate the use of web development, **machine learning**, and **database techniques**. The techniques will involve using **HTML, CSS, JavaScript, and Bootstrap** to create the **frontend** dashboard. The **backend** and REST APIs will be created through the use of **Python and Flask**. Data processing and analysis will be done using **Pandas** while machine learning models will be created using **Scikit-learn and XGBoost**. Sales, inventory, food waste, and forecasting data will be **stored** in **MySQL**. The connection between the frontend, backend, and machine learning will be through **REST APIs**. **Visual Studio Code** will be the tool for development and **Git and GitHub** for **version control**.		
Programming methodology	☐ Structured System Analysis and Design Methodology ☒ Object Oriented Analysis and Design Methodology		
SDLC	Agile		

Student No	CL/HDCSE/CMU/133/40 / st20345351	**Full Name**	R. M. Nimansa Pansiluni Rathnayake

Roles & Responsibilities;

Project Manager and Backend Developer

Coordinates project activities, schedules and monitors team activities and deadlines, interacts with the supervisor and monitors the project's progress. In my role as the **Backend Developer,** I create the server-side functionality using **Python and Flask,** implement business logic and user authentication, integrate the backend with the **MySQL** and develop **REST APIs**.

Student No	CL/HDCSE/CMU/133/96 / st20345396	**Full Name**	H. D. Dewmi Nethmini

Roles & Responsibilities;

System Analyst & Database Developer

Analyses system requirements, identifies user needs, defines system functions and designs the system workflow. As the Database Developer, designs and manages the MySQL database, creates tables and relationships, develops queries and CRUD operations and manages sales, inventory, menu, ingredient, and food waste data to ensure accurate and reliable data storage.

Student No	CL/HDCSE/CMU/133/114 / st20345409	**Full Name**	**S. G. M. Senugi Yewanthi Gunasekara**

Roles & Responsibilities;

UI/UX Engineer & Frontend Developer

Designs the user interface and user experience, including layouts, navigation, forms and dashboard screens. As the Frontend Developer, develops responsive and user friendly interfaces using HTML, CSS, JavaScript and Bootstrap, creates charts and visualizations, and connects the frontend with the backend through REST APIs.

Student No	CL/HDCSE/CMU/133/127 / st20345416	**Full Name**	H. Sashini Madushi Pinto

Roles & Responsibilities;

Software Engineer & Machine Learning Developer

Contributes to the overall system development and develops the machine learning component using Python, Pandas, Scikit-learn and XGBoost. Prepares and processes data, develops and trains food demand forecasting models, evaluates model performance and generates predictions to support food preparation and inventory decisions. Ensures the machine learning component is correctly integrated with the overall system.

Student No	CL/HDCSE/CMU/133/130 / st20345419	**Full Name**	W.W.P. Himaya Poojani

Roles & Responsibilities;

QA Engineer & System Integration Developer

Tests the system to identify errors and ensure all functions work correctly. Responsible for API and system integration, ensuring that the frontend, Python/Flask backend, MySQL database and machine learning components work together correctly through REST APIs. Performs system and integration testing, identifies technical issues and supports their resolution.

System Functionality:

System Administrator

Function	Description
System Dashboard	Provides an overall view of sales, demand forecasts, inventory, food waste, alerts, and system activity.
User Account Management	Creates, updates, activates, and manages user accounts for authorized system users.
Role & Access Management	Controls user roles and permissions to ensure users can access only authorized system functions.
Menu Management	Adds, edits, updates, and manages food items available in the restaurant.
Recipe/BOM Management	Defines the ingredients and required quantities for each food item.
Restaurant Information Management	Maintains basic restaurant information used by the system.
System Settings & Alert Configuration	Manages basic system settings and alert-related configurations.
Sales & Revenue Monitoring	Allows the administrator to view sales performance and revenue information.
Food Demand Forecast Monitoring	Displays machine-learning-based food demand forecasts for monitoring and decision-making.
Inventory Demand Forecast Monitoring	Displays predicted inventory requirements based on system forecasting.
Food Waste Monitoring	Allows the administrator to monitor recorded food waste and related trends.
Inventory Monitoring	Provides an overall view of inventory levels and stock-related information.
System Analytics	Provides analytical information about sales, demand, inventory, and waste.
System Reports	Provides access to sales, demand, inventory, waste, and other system reports.

Food Production Manager

Function	Description
Kitchen Dashboard	Displays daily food preparation requirements, predicted demand, ingredient requirements, stock information, and important alerts.
Daily Order Management	Allows the manager to view and manage daily food orders.
Pre-Order Management	Allows confirmed customer pre-orders to be viewed and considered when planning food preparation.
Food Demand Forecast	Displays predicted demand for each food item generated by the machine learning model.
Preparation Recommendations	Provides recommended quantities of each food item to prepare based on predicted demand.
Food Preparation Management	Supports planning and managing daily food preparation activities.
Preparation Quantity Recording	Records the actual quantity of each food item prepared.
Ingredient Requirements	Shows the ingredients and quantities required to meet the recommended preparation plan.
Ingredient Usage Recording	Records ingredients used during food preparation.
Actual Food Usage Recording	Records actual food usage during restaurant operations.
Sales/Served Quantity Recording	Records the actual portions sold or served for comparison with prepared and forecast quantities.
Food Waste Recording	Records leftover or wasted prepared food and its quantity.
Kitchen Stock Requests	Allows required ingredients to be requested from the inventory section.
Food Availability Management	Updates menu items as available or unavailable according to preparation and stock conditions.
Shortage Reporting	Reports insufficient ingredients or stock shortages to the Inventory & Procurement Manager.

Inventory & Procurement Manager

Function	Description
Inventory Dashboard	Provides an overview of current stock levels, predicted requirements, low-stock alerts, reorder recommendations, and inventory status.
Ingredient Management	Adds, updates, and maintains ingredient information used by the restaurant.
Stock Management	Manages available stock quantities and inventory-related records.
Stock-In Recording	Records ingredients received into inventory from suppliers or other sources.
Stock-Out Recording	Records ingredients issued from inventory, including stock supplied to the kitchen.
Stock Level Monitoring	Displays the current quantity of each ingredient available in inventory.
Minimum Stock Level Management	Defines minimum acceptable stock quantities for ingredients.
Reorder Level Management	Sets the stock level at which an ingredient should be considered for replenishment.
Ingredient Demand Forecast	Displays predicted future ingredient requirements based on demand forecasts.
Low-Stock Alerts	Generates alerts when ingredient quantities fall below defined stock levels.
Reorder Recommendations	Recommends which ingredients should be purchased and the required quantities.
Supplier Management	Maintains supplier details used for purchasing and replenishment activities.
Purchase Order Management	Creates and manages purchase orders for required ingredients.
Expiry Tracking	Tracks ingredient batches and expiry dates to reduce avoidable inventory waste.
Inventory Wastage Recording	Records expired, damaged, spoiled, or otherwise wasted ingredients.

Stock Adjustment Management	Records corrections and adjustments when actual stock differs from system records.
Kitchen Stock Request Management	Receives and processes ingredient requests submitted by the Food Production Manager.
Inventory Data Validation	Checks inventory-related records for accuracy and consistency.

Official use only;

Approval Status: **Approved / Rejected**

Date of the Status: ……..

Status: Approved / Rejected

Date of the status: ……

Approved by:

…… …….. …….. ……

1. Introduction

Imagine an ordinary Friday evening in any fast food restaurant in Sri Lanka. As the dinner rush begins to gather pace, chefs base their past experience and make arrangements for producing more of popular items such as hamburgers, fried chicken and French fries. However the demand does not always fit the expected pattern. By the time the business closes for the night, some of these foods and also ingredients that remain unused will become surplus which cannot be stored and reused and become waste.

The opposite situation may happen equally often. Demand for a certain type of food product may increase unexpectedly and before the kitchen realizes the situation there will be no stock available to meet it. There will be a stock out no available menu item an unhappy customer and a lost sale. Both situations highlight the same issue related to the operations of any restaurant how much food should be made and how much stock should be kept prior to knowing what the actual demand would be. In reality this is more likely to be determined by experience than analysis. Managers rely on past sales numbers, approximations and intuition. This is not necessarily a bad approach, but it does not take into account the degree of volatility of food demand daily, weekly, seasonal due to weather conditions, holidays, marketing campaigns etc. The fact that most of the products used in restaurants are perishable means that there is an immediate high cost associated with the mistake. Over provisioning leads to excess cost and wastage, under provisioning leads to lost customers. This is essentially a balancing act between two extremes.

The key to solving this problem lies in the fact that restaurants have all the necessary information to solve this problem at their disposal.The patterns within this data which items sell together how demand shifts around holidays which days consistently run short are often too subtle or too spread across records to spot through manual review or a basic spreadsheet. This is precisely the kind of problem machine learning is well suited to: learning from historical and contextual data to produce a usable estimate of future demand, rather than relying on last week's sales figure as a proxy for next week's. A forecast on its own, though, is not a solution. Its value depends on whether it can be turned into something a restaurant manager can act on. Linking a demand forecast to current inventory levels and recipe or ingredient requirements makes it possible to move from a raw prediction to a concrete recommendation how much of an item to prepare, which ingredients are likely to run low, and when a reorder should be

triggered. It is this translation from forecast to decision that separates a forecasting exercise from a genuinely useful decision support tool. While machine learning has already been applied to demand forecasting and inventory problems in various industries and regions, Sri Lankan fast food restaurants operate under conditions of their own distinct customer behaviour, a different holiday calendar, local weather patterns, promotional practices and varying levels of data availability and record-keeping maturity. Findings from other markets do not automatically transfer to this context, which leaves an open question worth investigating how can machine-learning-based forecasting and inventory decision support be meaningfully applied within Sri Lankan fast-food operations?

This research addresses that question through the development of SmartRestaurant, a Machine-Learning-Based Demand Forecasting and Inventory Decision Support System for Sri Lankan Fast-Food Restaurants. The proposed web-based system draws on historical and contextual operational data to forecast demand at the level of individual food items. These forecasts then feed into practical recommendations, how much to prepare, what ingredient quantities to expect, where low-stock situations are likely to arise, and when reordering should take place. The system is also designed to support the day-to-day recording of sales, inventory, food waste, and preparation quantities, so that the data required to keep improving these forecasts continues to be captured. The aim of this research, then, is twofold: to establish whether machine learning can produce reliable food-demand forecasts in this setting, and, just as importantly, to examine how those forecasts can be converted into decision support that restaurant staff will actually use. The intended outcome is a system that helps reduce unnecessary food waste and stock-outs while improving inventory management and the consistent availability of menu items. The remainder of this report is structured to reflect this research process. It covers the background and research problem, the research aim and objectives, a review of relevant literature, system requirements, the research and development methodology, the design of the proposed system and its machine learning approach, the implementation process, testing and evaluation, results and discussion, limitations of the study, and finally conclusions and recommendations.

2. Background study

Sri Lanka's fast-food sector has grown quickly over the past decade, fuelled by urbanisation, rising incomes, and a shift toward convenient dining among young professionals and students. International chains now compete alongside a large number of local outlets, especially in cities like Colombo, Kandy, and Galle. This growth has made competition tighter, and restaurants are under more pressure to run efficiently without sacrificing food quality or availability.

What makes fast food different from most other retail businesses is timing. Food is perishable and has to be prepared before demand is actually known. A batch of fried chicken or burger patties might only stay fresh for a few hours. So every restaurant is essentially guessing, then living with the consequences of that guess.

Two problems tend to follow from this. The first is food waste, caused mainly by over-preparation, unsold food and unused ingredients pile up by closing time and can't be reused. The second is stock-outs, where demand outpaces what was prepared and customers are turned away. Waste feels more visible because something physical gets thrown out, but stock-outs may actually hurt more in the long run, since a customer who can't get what they want doesn't just cost one sale, they may not come back.

Both problems come from the same source: restaurants struggle to estimate demand accurately. Most rely on past sales figures, rough rules of thumb, or a manager's gut feeling built from experience. This works reasonably well some of the time, but it breaks down when several factors overlap, a rainy Friday during school holidays, for example and it doesn't scale across a full menu of items. It also gives no clear way to turn a rough estimate into an actual preparation quantity or reorder decision.

Machine learning has already shown, in retail and e-commerce, that it can pick up on demand patterns humans tend to miss, especially when seasonality, promotions, or weather are involved. A smaller amount of research has applied this specifically to restaurants and food service, generally with promising results for reducing waste and improving stock management.

Very little of that work, however, looks at Sri Lanka specifically. Local restaurants deal with their own holidays, weather patterns, promotional habits, and in many cases, inconsistent digital record-keeping. Findings from other countries don't automatically apply here. On top of that, a lot of existing research stops at forecasting accuracy, it doesn't address how a forecast actually becomes a decision a manager can use, like how many patties to prepare tomorrow.

This gap, both the lack of Sri Lanka-specific research and the disconnect between forecasting and real operational decisions, is what this study responds to. The proposed SmartRestaurant system aims to bridge that gap by building a forecasting and decision-support tool designed around how Sri Lankan fast-food restaurants actually operate.

3. Problem statement

In particular, the business process that is the focus of the present research is the choice of products to be prepared by a fast-food restaurant and amounts of these products, as well as the management of raw materials inventory needed for their preparation. Often, such decisions are made on the basis of current sales statistics, experience, and managerial intuition without any special forecasting procedure. It is difficult in this case to take into account changing demand patterns and to transform expectations about the demand into preparation and inventory amounts.

Such an absence in the process leads to two recurrent results.If there is an overestimation of the demand for a product, there is an overproduction of the product, and all the perishable products such as fried chicken, hamburger meat, and cut-up vegetables left in stock after closing time have to be disposed of because they are no longer edible and could harm customers. The cost of the ingredients used in the preparation of the food and the labor used is lost.

In case of an underestimation of the demand for a product, the product sells out prematurely, and the customers are turned away or sold other products. Unlike waste, this cost is less visible, no physical stock is thrown away, but it is arguably more damaging over time, since an unavailable item can affect not just one sale but a customer's willingness to return. Both outcomes stem from the same underlying issue: preparation and inventory decisions are made without a reliable way of estimating demand in advance. This is not a failure of effort on the part of restaurant staff, but a limitation of the process itself. Experience and judgement can account for obvious patterns, such as weekends generally being busier than weekdays, but they struggle to account for the combined effect of multiple factors at once (for example, a promotional period falling during poor weather), and they offer no structured method for converting a demand estimate into a specific preparation quantity or ingredient order. At the same time, restaurants already generate the data that could support better decisions, daily sales records and basic inventory or stock information, but this data is generally used for record-keeping rather than for forecasting or decision support. It is collected, but not analysed in a way that feeds back into planning. Left unaddressed, this problem is likely to persist regardless of individual staff experience, since it is rooted in the absence of a structured, data-driven forecasting and decision-support process rather than in any specific person's judgement. This is the problem that the proposed SmartRestaurant system is intended to address, by using existing operational data to forecast demand and support more informed food preparation and inventory decisions.

4. Literature Review

Academic research. Arunraj and Ahrens (2015) developed SARIMA/SARIMAX models with quantile regression to forecast daily banana sales at a German discount retailer, outperforming several benchmarks. This shows statistical forecasting can add value for perishable goods, though the study is retail-based, not restaurant-based, and doesn't link forecasts to preparation decisions.

In a preprint paper from 2026 (Naeem et al.), ML models were trained on restaurant waste datasets, where Random Forest performed the best ($R^2 = 0.817$). This paper has yet to be peer-reviewed, and the authors clearly state that the waste variable used in the analysis is assumption-based and not ground truth, so it should be taken as indicative of results, but not established findings.

Apart from the above-mentioned two papers, the forecasting literature indicates that ML/hybrid methods may yield better results when there are several interacting context variables involved.

This should be read as a general tendency rather than a proven universal advantage, given the limited number of restaurant-specific studies available. Commercial systems. Toast's documented inventory functionality uses recipe-based stock depletion and par levels to identify low-stock items and support reorder decisions (Toast Support Center, n.d.). This goes beyond basic sales tracking, but it is fundamentally a rules-based approach: depletion is calculated from known recipes and current stock, and reorder points are triggered once stock falls below a set par level, rather than being predicted from contextual variables such as weather, promotions, or local holidays. The suitability and affordability of such commercial platforms for smaller Sri Lankan fast-food restaurants also requires further investigation, as this was not established by the sources reviewed. Gap. Based on the literature and systems reviewed, limited evidence was identified of solutions combining ML-based demand forecasting with ingredient-level decision support for Sri Lankan fast-food restaurants. This is the gap SmartRestaurant addresses.

5. Proposed solution

This study recommends **SmartRestaurant** as an online and machine learning driven decision support system that solves the problem described in the Problem Statement by analyzing existing data in the form of sales and stock of the restaurant to provide forecasts on demand for each particular product of the menu.

Structure. The structure of the system includes the following steps in the operation process: collection of data, demand forecasting, conversion of forecasts into actions, logging of outcomes, and analysis of outcomes. There are three main components of the system: **demand forecasting engine** (uses historical sales data, day of the week, holidays, weather,

and promotional data), **decision support layer** (provides recommendations on preparation of products based on recipes), and **outcomes logging layer** (allows the employees of the restaurant to log their sales, wastage, and stock levels). **Solving the problem.** The idea of item-level forecasting is to ensure that the amount of preparation matches demand prediction. Accounting for promotions, weather, and day-of-week patterns is intended to help identify potential demand changes before stock-outs occur. Unlike several academic studies reviewed that focus primarily on forecast accuracy, and compared with the commercial systems reviewed, which manage stock via recipe-based depletion and fixed par levels, SmartRestaurant links the forecast directly to an actionable recommendation. The forecasting engine is built around realistically obtainable data, historical sales plus available contextual information, reflecting the digital record-keeping level typical of Sri Lankan fast-food restaurants. **Key features:** item-level demand forecasting, preparation recommendations, ingredient requirement estimation, low-stock and reorder alerts, sales/waste/inventory logging, and a simple dashboard for non-technical staff. **Justification.** Commercial systems reviewed offer inventory management (tracking, recipe-based depletion, reorder support); academic studies reviewed focus mainly on forecasting performance. SmartRestaurant investigates how ML-based forecasting can connect directly to ingredient-level decision support within the Sri Lankan fast-food context, giving staff a data-driven, forward-looking basis for decisions they already have to make daily.

6. Justification to solution

These shortcomings were highlighted in the Problem Statement in the following way: firstly, food preparation and ordering decisions are based on managerial estimation and recent sales numbers rather than on an actual forecast and this results in over-preparation (waste) and under-preparation (stockouts). There are several arguments in support of the use of the SmartRestaurant in addressing the above problem.

Firstly, it provides structured estimation of demand. Instead of being based on managerial estimation which was noted in the Problem Statement to have a problem of taking into account the various factors (for example, promotions while the weather is bad) at once, the SmartRestaurant makes use of the forecasting model which is based on historical and contextual information and provides a structured estimation of demand. It does not solve the problem of uncertainty, but provides managers with a structured estimation which does not depend on whose turn to work it is.

It attacks both failure modes at once. As the literature review indicated, waste reduction and stock out prevention were traditionally handled as distinct failure modes, using distinct methodologies, software solutions for tracking and notifying stock levels, as well as academic forecasting models for predicting demand without doing anything about it. Because SmartRestaurant links a single forecast to both a preparation quantity and a

reorder recommendation, it is designed to address over-preparation and under-preparation as two sides of the same underlying estimation problem, rather than trading one off against the other. It closes the gap between forecasting and action identified in the literature. Several academic studies reviewed focus primarily on forecast accuracy without addressing how a prediction becomes an operational decision. SmartRestaurant's decision-support layer, which converts forecasts into ingredient-level preparation and ordering guidance, is intended to directly address this limitation, making the forecast usable at the point where preparation decisions are actually made. It builds on, rather than duplicates existing commercial capability. Compared with the commercial systems reviewed, which manage inventory through recipe-based depletion and fixed par levels SmartRestaurant does not attempt to replace established stock-tracking functionality. Instead, it adds a predictive layer, using contextual variables such as day-of-week, holidays, weather, and promotions, that were not identified in the documented functionality of the commercial systems reviewed, positioning it as a complementary rather than competing approach. It is designed around locally available data. Since the research identified a lack of Sri Lankan-specific study and questioned the fit of existing commercial platforms for local outlets, SmartRestaurant's forecasting engine is deliberately built around data that a typical local fast-food restaurant can realistically capture, historical sales and basic contextual information, rather than assuming the scale of data or existing digital infrastructure typical of larger international chains. It supports its own improvement over time. By recording actual sales, waste, and stock outcomes alongside generated forecasts, the system creates a structured dataset that can support future model evaluation and refinement. Taken together, these points justify SmartRestaurant not as a replacement for existing restaurant management practice, but as a targeted response to a specific, evidenced gap: the limited evidence identified in the reviewed literature and systems of solutions that connect machine-learning-based demand forecasting to practical, ingredient-level preparation and inventory decisions within the Sri Lankan fast-food setting. The actual extent to which it mitigates waste and stock-outs will depend on the forecasting accuracy achieved and will be assessed through the testing and evaluation described later in this report.

7. Scope of system

7.1 Scope of Work

SmartRestaurant is an online decision support application meant for the staff of fast food restaurants in Sri Lanka. The areas covered by SmartRestaurant include: predicting the demand for menu items in the specified short term future period, conversion of the forecast to preparation quantity, estimation of the quantity of ingredients from the recipes,

identification of potential shortage of stocks, documentation of actual sales/waste/stocks for evaluation and presenting all of the above in an intuitive dashboard interface.

7.2 Exclusions

Out of scope: multi-branch or chain-wide management; direct integration with external supplier systems, including automatic order placement, supplier pricing integration, and delivery tracking; POS and payment processing; staff scheduling; financial accounting; and continuous automated model retraining. Model updates will be performed periodically rather than automatically. Internal records such as supplier details and purchase order tracking remain within scope; it is only the automated exchange of this data with external supplier systems that is excluded.

7.3 Justification for Scope

This scope directly targets the single-outlet forecasting-to-decision gap identified in the Problem Statement and Literature Review, rather than broader supply-chain management. It also matches the realistic data and infrastructure typical of Sri Lankan fast-food outlets, keeping data requirements achievable. Finally, it keeps the project's functionality focused enough to be properly built and evaluated within an HND timeframe, rather than spread across too many features.

7.4 Functional Requirements

ID	Requirement
FR1	Authorised login with role-based access (System Administrator, Food Production Manager, Inventory & Procurement Manager)
FR2	Import/enter historical sales data per menu item
FR3	Record/import contextual data (day of week, holidays, promotions, weather)
FR4	Generate item-level demand forecasts for a defined short-term future period
FR5	Convert forecasts into recommended preparation quantities
FR6	Estimate ingredient requirements from forecasts and recipe data
FR7	Compare forecasted requirements to current stock and flag shortages
FR8	Record actual sales, preparation quantities, and waste

FR9	Record stock movements, including stock-in, stock-out, adjustments, and wastage
FR10	Display forecasts, recommendations, and alerts on a dashboard
FR11	Store forecasts and outcomes for future evaluation
FR12	View basic operational reports, including waste and inventory trends

7.5 Non-Functional Requirements

ID	Requirement	Description
NFR1	Usability	Simple enough for non-technical staff
NFR2	Performance	Forecasts generated within a few seconds
NFR3	Reliability	Handles missing/incomplete data gracefully
NFR4	Accessibility	Usable via standard browsers, desktop and mobile
NFR5	Security	Authentication and role-based access control
NFR6	Maintainability	Model and system logic structured for periodic updates
NFR7	Scalability	System architecture and data structures should support possible future extension to multiple outlets
NFR8	Data Integrity	Input validation to prevent forecast distortion

This scope keeps the system deliberately bounded to a single restaurant's forecasting and preparation/inventory decisions, leaving external supplier integration, financial accounting, and multi-branch functionality for future work. Forecast accuracy itself is addressed separately in the Testing and Evaluation section, using measures such as MAE, RMSE, MAPE, or R^2 depending on the final model evaluation approach.

8 Methodology & Technology

8.1 Software Development Methodology

The Agile Software Development Life Cycle (SDLC) methodology will be used for the development of SmartRestaurant. Agile is suitable because the system consists of several

connected components, including the database, backend, frontend, machine learning model, and decision-support functions, which can be developed and tested incrementally.

The development process will include requirements analysis, system design, data preparation, implementation, machine learning model development, system integration, testing, evaluation, and refinement. Each component will be reviewed against the defined system requirements, allowing issues and changes to be addressed throughout development rather than only after the complete system has been developed.

Object-Oriented Analysis and Design (OOAD) will be used to support system analysis and design. UML diagrams such as Use Case, Class, Activity, and Sequence Diagrams will be used where appropriate to represent users, system processes, objects, relationships, and interactions.

8.2 Programming Languages and Frameworks

HTML, CSS, JavaScript, and Bootstrap will be the primary technologies used for creating a responsive web interface for the frontend of SmartRestaurant. This interface will be used for presenting forecasts, preparation recommendations, inventories, alerts, and reporting.

Python is the programming language that will be utilized for the backend development, data processing, and machine learning. Flask will be used as the backend web framework and to develop REST APIs that support communication between system components. Pandas will be used for cleaning, transforming, and preparing data. Machine learning models will be developed and evaluated using Scikit-learn and XGBoost to determine a suitable approach for demand forecasting.

8.3 Database Management System

MySQL will be used as the Database Management System (DBMS). It will store structured system data including users, menu items, recipes, ingredients, sales, inventory transactions, preparation quantities, food waste, forecasts, and recommendations.

8.4 Software and Hardware Tools

Category	Tool / Requirement	Purpose
Development Environment	Visual Studio Code	Coding and system development
Version Control	Git / GitHub	Source control and team collaboration
Backend	Python / Flask	Backend logic and REST APIs

Frontend	HTML, CSS, JavaScript, Bootstrap	Responsive web interface
Data Processing	Pandas	Data cleaning and preparation
Machine Learning	Scikit-learn, XGBoost	Forecasting model development and evaluation
DBMS	MySQL	System data storage and management
Testing Browser	Chrome / Firefox / Edge	Web application testing
Development Hardware	Laptop/Desktop, 8GB+ RAM, multi-core CPU, 10GB+ free storage	Development and ML processing
Deployment Hardware	Basic cloud hosting suitable for Flask and MySQL	System deployment/demo

8.5 Cost Requirements

Since SmartRestaurant is an academic project and primarily uses free and open-source technologies, software licensing costs are expected to be minimal.

Resource	**Estimated Cost (LKR)**	**Notes**
Development Software	Free	Most selected development tools are free/open-source
Hosting / Deployment	To be confirmed	Depends on selected hosting provider
Domain Name	To be confirmed	Required only if a public domain is used
Internet / Data	To be confirmed	Development, research and testing
Data Collection / Transport	To be confirmed	If restaurant visits/data collection are required

Printing / Documentation	To be confirmed	Final documentation and submission
Contingency	To be confirmed	Unexpected project expenses

8.6 Hardware and Software Requirements

Type	Requirements
Hardware	Laptop/desktop computer, minimum 8GB RAM, multi-core processor, 10GB+ free storage and internet connection
Operating System	Windows, macOS or Linux
Programming Languages	Python, JavaScript
Frontend Technologies	HTML, CSS, JavaScript, Bootstrap
Backend Technologies	Python, Flask, REST APIs
ML/Data Libraries	Pandas, Scikit-learn, XGBoost
Database	MySQL
Development Tools	Visual Studio Code, Git, GitHub
Testing Tools	Chrome, Firefox or Edge
Deployment	Cloud environment capable of running Python/Flask and MySQL

9. Gantt chart

#	Project Activity	Responsible	Sep Week 1	Sep Week 2	Sep Week 3	Sep Week 4	Oct Week 1	Oct Week 2
REQUIREMENTS & SYSTEM DESIGN								
1	Finalise Requirements & Use Cases (Pilot Restaurant)	Dewmi						
2	Database Design – Sales, Inventory & Contextual Data Schema	Nimansa						
3	Dashboard UI/UX Design & Prototyping	Senugi						
SERVICE DEVELOPMENT								
4	Data Management Service (Sales Import & Contextual Data)	Nimansa						
5	Demand Forecasting Service – ML Model (Random Forest, XGBoost)	Sashini						
6	Inventory Service – Stock & Shelf-Life Tracking	Nimansa						
7	Recommendation Service – Prep & Purchasing Logic	Sashini						
8	Alert / Notification Service – Stock-Out & Waste-Risk Alerts	Sashini						
9	Dashboard Development & API Integration	Senugi						
INTEGRATION & TESTING								
10	System Integration Across All Six Services	Nimansa, Senugi, Sashini						
11	QA Testing – Forecast Accuracy & Recommendation Checks	Himaya						

#	Project Activity	Responsible	Sep Week 1	Sep Week 2	Sep Week 3	Sep Week 4	Oct Week 1	Oct Week 2
12	Usability Testing & User Acceptance Testing (Pilot Restaurant)	Himaya						
FINALISATION								
13	Final Documentation & Report Completion	Nimansa, Dewmi						
14	Final Presentation / Viva Preparation	All Members						

Legend

	Requirements & System Design		Service Development		System Integration		Testing (QA / UAT)		Finalisation

10.Risk

Risk / Uncertainty	Possible Impact	Mitigation
Insufficient historical sales data	The ML model may not produce reliable demand forecasts	Collect sufficient historical or representative data before model development
Missing or inaccurate data	May reduce forecasting accuracy	Clean, validate and preprocess data before model training
Limited contextual data	Weather, holiday or promotion factors may not always be available	Use available contextual variables and document unavailable data as a limitation
Poor ML model performance	Forecasts may not be accurate enough for decision support	Develop, test and compare multiple models before selecting the most suitable model
System integration issues	Frontend, Flask backend, MySQL and ML components may not work correctly together	Integrate components gradually and perform integration testing
Development delays	Activities may fall behind the planned schedule	Monitor progress regularly and adjust task allocation when required
Team member unavailability	Assigned development activities may be delayed	Use shared documentation, GitHub and clear task allocation

Software defects	System functions may fail or produce incorrect results	Conduct functional, integration and system testing throughout development
Security risks	Unauthorised access or incorrect handling of system data	Apply authentication, role-based access control and input validation
Scope creep	Additional features may increase development time and complexity	Follow the approved project scope and leave non-essential features for future development
Deployment issues	The system may not operate correctly during the final demonstration	Test deployment in advance and maintain a working local version as backup